蓝桥杯按键考点

1. 按键底层

1.1 基础按键底层

```c
unsigned char Key_Read()
{
    unsigned char temp = 0;
    P44 = 0;    P42 = 1;  P35 = 1;    P34 = 1;
    if (P33 == 0)    temp = 4;
    if (P32 == 0)    temp = 5;
    if (P31 == 0)    temp = 6;
    if (P30 == 0)    temp = 7;
    P44 = 1;    P42 = 0;    P35 = 1;    P34 = 1;
    if (P33 == 0)    temp = 8;
    if (P32 == 0)    temp = 9;
    if (P31 == 0)    temp = 10;
    if (P30 == 0)    temp = 11;
    P44 = 1;    P42 = 1;    P35 = 0;    P34 = 1;
    if (P33 == 0)    temp = 12;
    if (P32 == 0)    temp = 13;
    if (P31 == 0)    temp = 14;
    if (P30 == 0)    temp = 15;
    P44 = 1;    P42 = 1;    P35 = 1;    P34 = 0;
    if (P33 == 0)   temp = 16;
    if (P32 == 0)    temp = 17;
    if (P31 == 0)    temp = 18;
    if (P30 == 0)    temp = 19;
    return temp;
}
```

Fence 1

> 这个是最基础的底层，通过下面的原理图看出来的（这里使用最新的十五届原理图），这里就不过多赘述了，通过检测低电平来进行判断

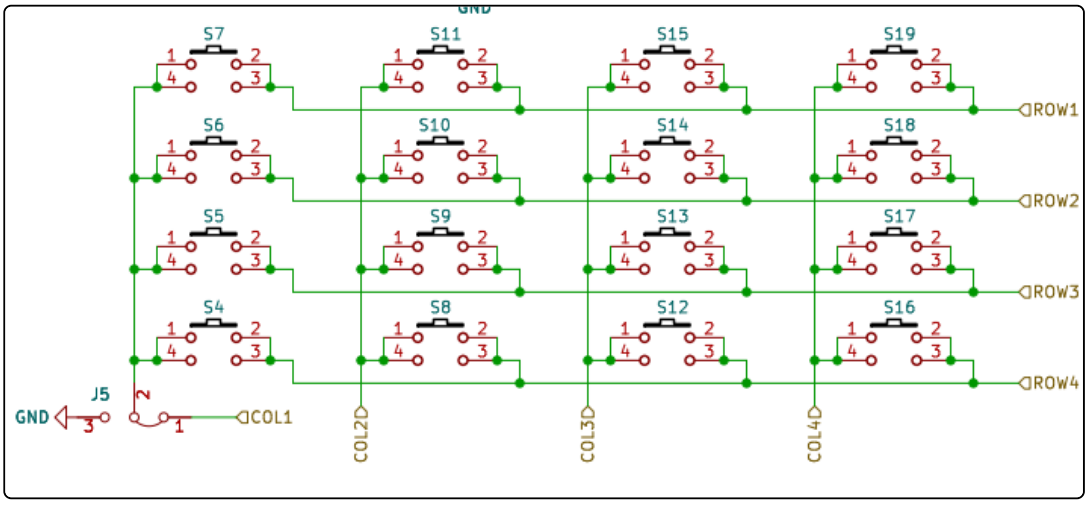


Image 1

1.1.1 特殊情况（独立按键）

独立按键只会用到最左边那一列（注意板子上需要切换为btn模式）

```c
unsigned char Key_Read()
{
    unsigned char temp = 0;
    if (P33 == 0)    temp = 4;
    if (P32 == 0)    temp = 5;
    if (P31 == 0)    temp = 6;
    if (P30 == 0)    temp = 7;
    return temp;
}
```

Fence 2

1.1.2 特殊情况（串口使用）

串口通信使用P30和P31引脚，这两个引脚同时也用于矩阵键盘扫描。为确保两个功能正常协同工作，需遵循以下规则：

- **必须采用列扫描方式**：因P30和P31已被串口占用，矩阵键盘只能进行列扫描
- **禁止直接操作P30/P31**：键盘扫描过程中不得对P30和P31赋值，以防串口发送错误数据
- **扫描前检测P30状态**：每次扫描前必须检测P30是否为高电平（因P30作为接收引脚Rx时，接收数据会使其拉低，可能导致误判）
- **删掉无用按键**：删除掉无用的按键，可以降低触发概率

```c
#include "key.h"
unsigned char Key_Read()
{
    unsigned char temp = 0;

    if(P30 == 0) return 0;
    P44 = 0;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) temp = 7;
    if(P31 == 0) temp = 6;
    if(P32 == 0) temp = 5;
    if(P33 == 0) temp = 4;
    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 0;P35 = 1;P34 = 1;
    if(P30 == 0) temp = 11;
    if(P31 == 0) temp = 10;
    if(P32 == 0) temp = 9;
    if(P33 == 0) temp = 8;
    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 1;P35 = 0;P34 = 1;
    if(P30 == 0) temp = 15;
    if(P31 == 0) temp = 14;
    if(P32 == 0) temp = 13;
    if(P33 == 0) temp = 12;
    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 1;P35 = 1;P34 = 0;
    if(P30 == 0) temp = 19;
    if(P31 == 0) temp = 18;
    if(P32 == 0) temp = 17;
    if(P33 == 0) temp = 16;
    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    P3 = P3|0xef;
    return temp;
}
```

Fence 3

1.1.3 特殊情况（NE555）

我们这里会遇到NE555的频率测量，我们使用P34链接signal引脚进行测量，但是我们要注意，因为我们NE555测量本质是对边沿进行计数，所以我们这里不能使用P34作为按键的大前提（手动赋值）和判断依据（if里面），这是硬件的冲突，没法避免，所以我们把和P34有关的所有数据全删了。

```
unsigned char Key_Read()
{
    unsigned char temp = 0;
    P44 = 0;    P42 = 1;    P35 = 1;
    if (P33 == 0)    temp = 4;
    if (P32 == 0)    temp = 5;
    if (P31 == 0)    temp = 6;
    if (P30 == 0)    temp = 7;
    P44 = 1;    P42 = 0;    P35 = 1;
    if (P33 == 0)    temp = 8;
    if (P32 == 0)    temp = 9;
    if (P31 == 0)    temp = 10;
    if (P30 == 0)    temp = 11;
    P44 = 1;    P42 = 1;    P35 = 0;
    if (P33 == 0)    temp = 12;
    if (P32 == 0)    temp = 13;
    if (P31 == 0)    temp = 14;
    if (P30 == 0)    temp = 15;
    return temp;
}
```

Fence 4

1.1.4 特殊情况（串口使用+NE555使用）

这里主要就是屏蔽P34

```
#include "key.h"
unsigned char Key_Read()
{
    unsigned char temp = 0;

    if(P30 == 0) return 0;
    P44 = 0;P42 = 1;P35 = 1;
    if(P30 == 0) temp = 7;
    if(P31 == 0) temp = 6;
    if(P32 == 0) temp = 5;
    if(P33 == 0) temp = 4;
    P44 = 1;P42 = 1;P35 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 0;P35 = 1;
    if(P30 == 0) temp = 11;
    if(P31 == 0) temp = 10;
    if(P32 == 0) temp = 9;
    if(P33 == 0) temp = 8;
    P44 = 1;P42 = 1;P35 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 1;P35 = 0;
    if(P30 == 0) temp = 15;
    if(P31 == 0) temp = 14;
    if(P32 == 0) temp = 13;
    if(P33 == 0) temp = 12;
    P44 = 1;P42 = 1;P35 = 1;;
    P3 = P3|0xef;
    return temp;
}
```

Fence 5

1.2 应用层（三行代码消抖）

```
    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
```

Fence 6

> 这个代码其实在西风的文档里面单独列了一下，我这里将其放在一起来进行描述
>
> 瞬时值获取-> 获取按下的值，获取抬起的值->更新一下值
>
> Down和Up都是瞬时值，Old比Val延后10ms，基于这个特性我们可以进行书写代码

这里以S4（0100）为例，来进行说明

键盘状态	Key_Old	Key_Val	Key_Old^Key_Val	Key_Down	Key_Up
未按下	0000	0000	0000^0000=0000	0000	0000
按下过程中（10ms）	0000	0100	0000^0100=0100	0100&0100=0100	1011&0100=0000
按下稳定（10ms后）	0100	0100	0100^0100=0000	0100&0000=0000	1011&0000=0000
抬起过程（10ms）	0100	0000	0100^0000=0100	0000&0100=0000	1111&0100=0100

Table 1

> 我们可以从这里看出，当我们按下的时候，那一瞬间Down会是4，其余时候都是0；同理Up也是一样，当我们抬起的瞬间Up会是4，其余时候是0

1.3 调用

> 那么我们应该如何进行调用呢，其实很简单，如下所示

```
if (Key_Down == 4)
    //执行按下4的函数
```

Fence 7

2. 按键考点

2.1 模式切换

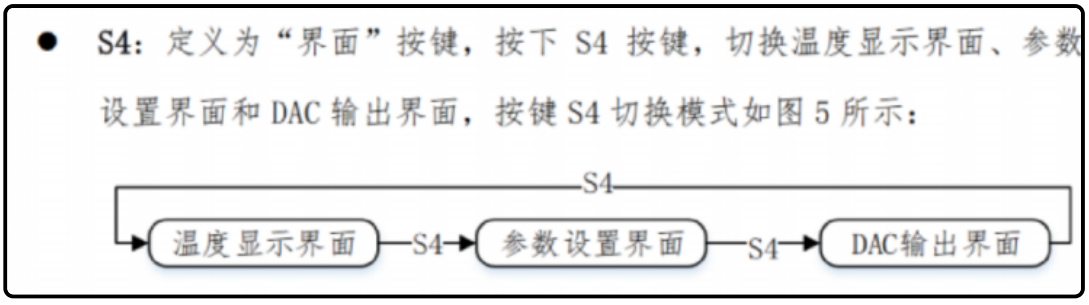


Image 2

```c
//以这个为例，我们有三个界面 按键考点-米醋工作室_左岚
//首先需要定义一个界面显示的一个变量
unsigned char Seg_show_mode;//0 温度显示页面 1 参数设置页面 2 DAC输出页面
//然后开始我们的键盘函数
void Key_Proc()
{
    static unsigned char Key_Val, Key_Down, Key_Up, Key_Old;
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
    if (Key_Down == 4)
        //0%3=0 1%3=1 2%3=2 3%3=0
        Seg_show_mode = ( ++ Seg_show_mode) % 3;
}
```

Fence 8

> 这里我们直接进行取余，就可以保证这个值在0-2之间循环

2.2 参数设置

- S8：定义为“加”按键，参数界面下，按下温度参数值加 1。
- S9：定义为“减”按键，参数界面下，按下温度参数值减 1；时间回显子界面下，长按 S9 超过 2 秒后松开，清除所有已记录的数据，触发次数重置为 0。

Image 3

> 这里我们不说他那个长按的问题，我们就看单纯的±参数，调用如下所示

```c
//以这个为例，我们有一个参数
//首先需要定义参数
unsigned char dat;//需要调整的参数
unsigned char seg_show_mode;//1为参数界面
//然后开始我们的键盘函数
void Key_Proc()
{
    static unsigned char Key_Val, Key_Down, Key_Up, Key_Old;
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
    //在参数界面下
    if(seg_show_mode==1)
    {
        //一定注意上下限
        if (Key_Down == 8)
            //假定dat的取值的上限是100
            dat = ( ++ dat > 100) ? 100 : dat;
        if (Key_Down == 9)
            //假定dat的取值下限是10
            dat = ( -- dat < 10) ? 10 : dat;
    }
}
```

Fence 9

> 这里其实要注意的点就是，一定要看清楚上下限，否则会导致超出扣分

2.3 参数保存

> 参数保存也是一个考点，主要是修改的数据是否实时生效，上面的参数设置就是实时生效，而这里的参数保存就是延迟生效，修改参数的时候，原本的参数不产生变化
>
> 例子银行转账
>
> - 真实值：账户实际余额
> - 显示值：手机银行输入框的数字
>
> 假设你有100元，想给朋友转50元。当你在输入框输入「5」时，如果实时扣款，系统会先扣5元，再扣50元，这显然不合理。所以需要等你输入完整金额并确认后，才真正修改真实余额。

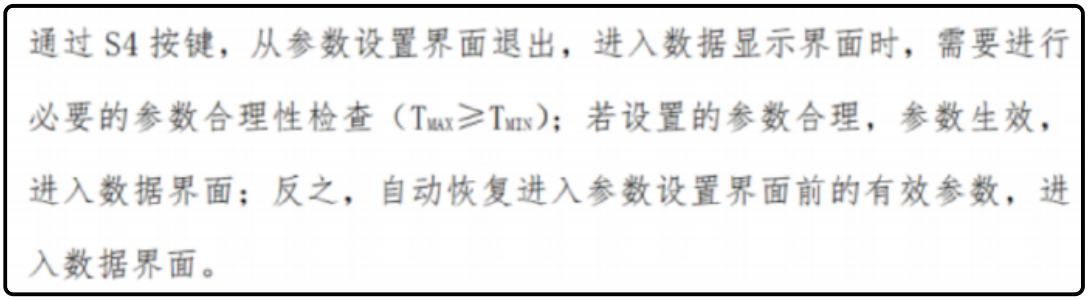


Image 4

```c
//以这个为例，我们有一个参数
//首先需要定义参数
unsigned char Tmax_Control,Tmin_Control;//参数设置值
unsigned char Tmax,Tmin;//参数保存值
//然后开始我们的键盘函数
void Key_Proc()
{
    static uchar Key_Val, Key_Down, Key_Up, Key_Old;
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
    //一定注意上下限
    if (Key_Down == 8)
        Tmax_Control ++ ;
    if (Key_Down == 9)
        Tmin_Control ++ ;
    //按下S4确认输入的数据
    if (Key_Down == 4)
    {
        //当我们满足条件
        if (Tmax_Control >= Tmin_Control)
        {
            Tmax = Tmax_Control;
            Tmin = Tmin_Control;
        }
        //不满足条件，值不进行修改，并且控制值复原为原本的值
        else
        {
            Tmax_Control = Tmax;
            Tmin_Control = Tmin;
        }
    }
}
```

2.4 按键长短按

这里也分为了三个情况

- 长按S4 1S，不用松手，执行**一次**LED的状态反转，短按S4，数据+1
- 长按S5 1S，不用松手，数据开始快速增加，短按S5，数据+1
- 长按S6 1S以上，松手执行数据+5，短按S6，数据+1

2.4.1 按键方面

```
unsigned int Time_1s_4, Time_1s_5, Time_1s_6;
bit Long_Press_4, Long_Press_5, Long_Press_6;
bit Led_Flag;
unsigned char my_data1, my_data2, my_data3;
bit Func_Run_Flag;//是否运行

void Key_Proc()
{
    static uchar Key_Val, Key_Down, Key_Up, Key_Old;
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val); // 按键按下检测
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);  // 按键抬起检测
    Key_Old = Key_Val;

    // 按下后不用抬起生效，并且只生效一次
    if (Key_Down == 4)
    {
        Long_Press_4 = 1;     // 按键4按下,开始长按检测
        Func_Run_Flag = 0; //程序未执行
    }
    if (Time_1s_4 >= 1000) // 到达时间节点，应该执行一次
    {
        //程序没有执行过
        if(Func_Run_Flag == 0)
        {
            Led_Flag ^= 1;
            Func_Run_Flag = 1;
        }
    }
    if (Key_Up == 4)
    {
        //短按
        if(Time_1s_4<1000)
            my_data1++;         // 短按+1
        Long_Press_4 = 0; // 按键4抬起，长按结束
        Time_1s_4 = 0;     // 重置时间
    }


    // 短按慢加，长按快加
    if (Key_Down == 5)
        Long_Press_5 = 1;
    if (Time_1s_5 >= 1000)
    {
        if (Key_Old == 5)
            my_data2++; // 长按一直+1
    }
    if (Key_Up == 5)
    {
        if (Time_1s_5 < 1000)
            my_data2++; // 短按+1
        Long_Press_5 = 0;
```

```
        Time_1s_5 = 0;
    }

    // 长按松手执行任务（数据+5）
    if (Key_Down == 6)
        Long_Press_6 = 1;
    if (Key_Up == 6)
    {
        if (Time_1s_6 >=1000)
            my_data3 += 5;
        else
            my_data3++;
        Long_Press_6 = 0;
        Time_1s_6 = 0;
    }
}
```

Fence 11

2.4.2 定时器方面

```
void Timer1_Isr(void) interrupt 3
{
    if (Long_Press_4)
    {
        if (++Time_1s_4 >= 1000)
            Time_1s_4 = 1001; // 卡死时间
    }
    else
        Time_1s_4 = 0; // 清零
    if (Long_Press_5)
    {
        if (++Time_1s_5 >= 1000)
            Time_1s_5 = 1001; // 卡死时间
    }
    else
        Time_1s_5 = 0; // 清零
    if (Long_Press_6)
    {
        if (++Time_1s_6 >= 1000)
            Time_1s_6 = 1001; // 卡死时间
    }
    else
        Time_1s_6 = 0; // 清零
}
```

Fence 12

2.5 按下跳转 松手返回

在“时钟显示”状态下，按下 S4 按键，显示温度数据，松开按键，返回“时钟显示”界面。

Image 5

> 这里其实很好解释，按下可以使用Old进行判定，松开可以通过Up进行判定，为什么我们不用Old进行松手判定呢，因为Old在没有按键按下的时候一直是0，所以会导致错误计算。

```
//以这个为例
//首先需要定义一个界面显示的一个变量
unsigned char Seg_show_mode;//0 时间显示 1 温度显示
//然后开始我们的键盘函数
void Key_Proc()
{
    static uchar Key_Val, Key_Down, Key_Up, Key_Old;
    if (Key_Slow_Down)
        return;
```

```
    Key_Slow_Down = 1;      按键考点-米醋工作室_左岚

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
    if(Key_Old==4)
        Seg_show_mode=1;
    if(Key_Up==4)
        Seg_show_mode=0;
}
```

Fence 13

2.6 密码门输入

这里主要是说明一下输入，即从右到左输入显示数据，并且高位熄灭

我们首先需要修改一下底层，适应我们的键盘，这里提醒一下，我们不能把原本的temp=0给改了，这个地方必须锁死

然后为了防止按键冲突，这里建议使用100-109来映射键盘0-9，并且使用一个不会使用的值（比如10）来作为输入缓存的初始化数据

键盘设定如下所示

S7=7->107	S11=8->108	S15=9->109
S6=4->104	S10=5->105	S14=6->106
S5=1->101	S9=2->102	S13=3->103
S4 验证密码	S8=0->100	S12 清空

Table 2

验证密码错误后，清空当前输入的密码

示例：
输入序列: 1,2,3
n = 3

Seg_Buf[7] = input_passwd[2] = 3
Seg_Buf[6] = input_passwd[1] = 2
Seg_Buf[5] = input_passwd[0] = 1
Seg_Buf[0]~Seg_Buf[4] = 熄灭

显示效果：_ _ _ _ _ 1 2 3

2.6.1 按键底层

```
unsigned char Key_Read()
{
    unsigned char temp = 0;  // 初始值必须为0，表示无按键按下

    // 第一行扫描（验证密码、数字1、4、7）
    P44 = 0;    P42 = 1;    P35 = 1;    P34 = 1;
    if (P33 == 0)    temp = 4;     // 验证密码按键
    if (P32 == 0)    temp = 101;   // 数字1按键
    if (P31 == 0)    temp = 104;   // 数字4按键
    if (P30 == 0)    temp = 107;   // 数字7按键
```

```
    // 第二行扫描（数字0、2、5、8）
    P44 = 1;    P42 = 0;    P35 = 1;    P34 = 1;
    if (P33 == 0)    temp = 100;   // 数字0按键
    if (P32 == 0)    temp = 102;   // 数字2按键
    if (P31 == 0)    temp = 105;   // 数字5按键
    if (P30 == 0)    temp = 108;   // 数字8按键

    // 第三行扫描（清空、数字3、6、9）
    P44 = 1;    P42 = 1;    P35 = 0;    P34 = 1;
    if (P33 == 0)    temp = 12;    // 清空按键
    if (P32 == 0)    temp = 103;   // 数字3按键
    if (P31 == 0)    temp = 106;   // 数字6按键
    if (P30 == 0)    temp = 109;   // 数字9按键

    // 第四行扫描（预留按键）
    P44 = 1;    P42 = 1;    P35 = 1;    P34 = 0;
    if (P33 == 0)    temp = 16;    // 预留按键
    if (P32 == 0)    temp = 17;    // 预留按键
    if (P31 == 0)    temp = 18;    // 预留按键
    if (P30 == 0)    temp = 19;    // 预留按键

    return temp;  // 返回按键值
}
```

Fence 14

2.6.2 按键应用层

```
unsigned char current_passwd[8] = {1, 2, 3, 4, 5, 6, 7, 8};  // 系统设定的正确密码
unsigned char input_passwd[8] = {10,10,10,10,10,10,10,10};   // 用户输入的密码缓存
unsigned char input_passwd_index = 0;                        // 当前输入密码的位置索引
bit passwd_error = 0;                                        // 密码错误标志位

// 密码键盘处理函数
void Key_Proc()
{
    static unsigned char Key_Val, Key_Down, Key_Up, Key_Old;

    // 按键防抖动处理
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    // 读取按键值并处理按键状态
    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);    // 检测按键按下
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);     // 检测按键释放
    Key_Old = Key_Val;

    // 数字键输入处理（密码未满8位时接受输入）
    if (input_passwd_index < 8)
    {
        // 检测数字键按下（100-109对应0-9）
        if (Key_Down >= 100 && Key_Down <= 109)
        {
            input_passwd[input_passwd_index] = Key_Down - 100;  // 转换为实际数字0-9
            input_passwd_index++;                               // 索引指向下一位
        }
    }

    // 验证密码按键处理
    if (Key_Down == 4)
    {
        // 验证输入的密码是否正确
        if (memcmp(input_passwd, current_passwd, 8) == 0)
        {
            // 密码正确处理
            passwd_error = 0;
            // 可在此添加开门等后续动作
        }
```

```
        else
        {
            // 密码错误处理
            passwd_error = 1;
            // 可在此添加错误提示等功能
        }
        // 无论正确与否，都清空当前输入
        input_passwd_index = 0;
        memset(input_passwd, 10, 8);
    }

    // 清空按键处理
    if (Key_Down == 12)
    {
        input_passwd_index = 0;      // 重置索引
        memset(input_passwd, 10, 8); // 清空输入缓存
    }
}

```

Fence 15

2.6.3 显示部分

```
void Seg_Proc()
{
    unsigned char i;

    // 显示刷新防抖处理
    if (Seg_Slow_Down)
        return;
    Seg_Slow_Down = 1;

    // 当有数字输入时进行显示处理
    if (input_passwd_index > 0)
    {
        // 从右向左显示输入的数字(右对齐)
        for (i = 0; i < input_passwd_index; i++)
        {
            // 数组索引映射：从最右边(7)开始，显示最近输入的数字
            Seg_Buf[7 - i] = input_passwd[input_passwd_index - i - 1];
        }

        // 未输入的高位数码管熄灭
        for (; i < 8; i++)
        {
            Seg_Buf[7 - i] = 10;  // 10代表段码全灭
        }
    }
    // 无数据输入时全部熄灭
    else
    {
        for (i = 0; i < 8; i++)
        {
            Seg_Buf[i] = 10;  // 所有数码管熄灭
        }
    }

}

```

Fence 16

2.7 双按键

S8、S9：定义“恢复出厂设置”功能。

若在任意界面下，检测到 S8、S9 按键均处于按下状态，且状态持续时间超过 2 秒，则恢复到**初始状态（四、初始状态）**。

> 这里需要修改一下底层，修改如下所示，新增temp=89，并且一定要放在最后的判断

2.7.1 底层代码

```
unsigned char Key_Read()
{
    unsigned char temp = 0;
    P44 = 0;    P42 = 1;  P35 = 1;    P34 = 1;
    if (P33 == 0)    temp = 4;
    if (P32 == 0)    temp = 5;
    if (P31 == 0)    temp = 6;
    if (P30 == 0)    temp = 7;
    P44 = 1;    P42 = 0;    P35 = 1;    P34 = 1;
    if (P33 == 0)    temp = 8;
    if (P32 == 0)    temp = 9;
    if (P31 == 0)    temp = 10;
    if (P30 == 0)    temp = 11;
    if (P33 == 0 && P32 == 0) temp = 89;
    P44 = 1;    P42 = 1;    P35 = 0;    P34 = 1;
    if (P33 == 0)    temp = 12;
    if (P32 == 0)    temp = 13;
    if (P31 == 0)    temp = 14;
    if (P30 == 0)    temp = 15;
    P44 = 1;    P42 = 1;    P35 = 1;    P34 = 0;
    if (P33 == 0)   temp = 16;
    if (P32 == 0)    temp = 17;
    if (P31 == 0)    temp = 18;
    if (P30 == 0)    temp = 19;
    return temp;
}
```

Fence 17

2.7.2 按键方面

> 这里判断一下就好了，注意，不要判断down，需要判断old，因为down只在那一瞬间有值

```

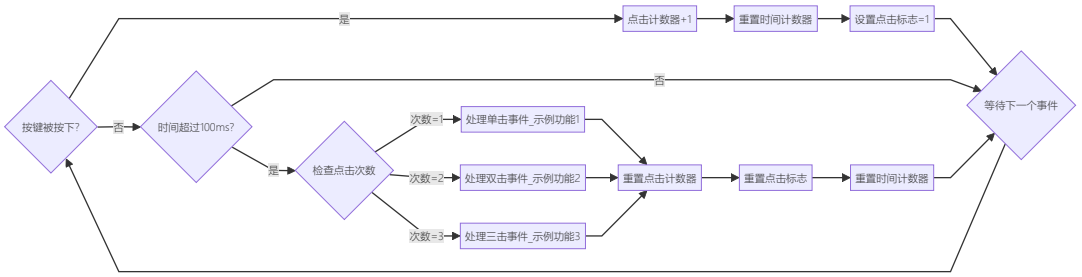
//然后开始我们的键盘函数
void Key_Proc()
{
    static unsigned char Key_Val, Key_Down, Key_Up, Key_Old;
    unsigned char i;
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
    if (Key_Old == 89)
    {
        // 双按键按下
    }
    if (Key_Up == 8)
    {
    }
    if (Key_Up == 9)
    {
    }
}

```

Fence 18

> 这个是群友提到的一个问题，这里做个简单说明，首先，我们要明确一下按键按下的阈值时间（我这里认为是100ms），然后就是进行一个计数就行
>
> 这里用S4来举例



2.8.1 按键方面

```c
//以这个为例
bit Key_click_flag;//按键触发标志
unsigned char time_100ms;
unsigned char Key_click_count;
//然后开始我们的键盘函数
void Key_Proc()
{
    static unsigned char  Key_Val, Key_Down, Key_Up, Key_Old;
    unsigned char i;
    if (Key_Slow_Down)
        return;
    Key_Slow_Down = 1;

    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Old ^ Key_Val);
    Key_Up = ~Key_Val & (Key_Old ^ Key_Val);
    Key_Old = Key_Val;
    if (Key_Down == 4)
    {
        Time_100ms = 0;
        Key_Click_Flag = 1;
        Key_Click_Count++;
    }
    if (Time_100ms >= 100)
    {
        switch (Key_Click_Count)
        {
            case 1:
                // 单击
                break;
            case 2:
                // 双击
                break;
            case 3:
                // 三击
                break;
        }
        Key_Click_Count = 0; // 清零，为后面做准备
        Key_Click_Flag = 0;
        Time_100ms = 0;
    }
}
```

Fence 19

2.8.2 定时器方面

```
void Timer1_Isr(void) interrupt 3
{
    if (Key_Click_Flag)
    {
        if (++Time_100ms >= 100)
            Time_100ms = 101; // 按键计时增加
    }
    else
        Time_100ms = 0; // 按键计时清零
}
```

Fence 20